



객체 감지를 탑재한 자율주차 시스템

Team 1

김소원 서영훈 전호준 진나영



About Team1

[프로젝트 목표]

1. 주차 공간 탐색
→ 특별 주차 공간 / 주차 불가 공간 구분
2. 공간 외 객체 감지
→ 주차 불가 표식 / 장애물 / 이동 물체 구분

[역할 분담]

공통 테스트		데이터셋 전처리, 모델 디자인 &
-----------	--	--------------------

이름		주요 역할
----	--	-------

김소원		발표 자료 제작
-----	--	----------

서영훈		YOLO 모델 성능 개선
-----	--	---------------

전호준		Segformer 모델 성능 개선, 영상 인식 프로그램 개선
-----	--	--------------------------------------

진나영		모델 구조 선정, 후처리 코드 작성, 모델 병합된 영상 인식 프로그램 작성
-----	--	--

About Team1



[학습 방법]

- 엘리스 플랫폼을 이용하여 회의
- 디스코드를 통해 상황 공유
- 깃랩, 노션을 통해 내용 공유

[학습 내용]

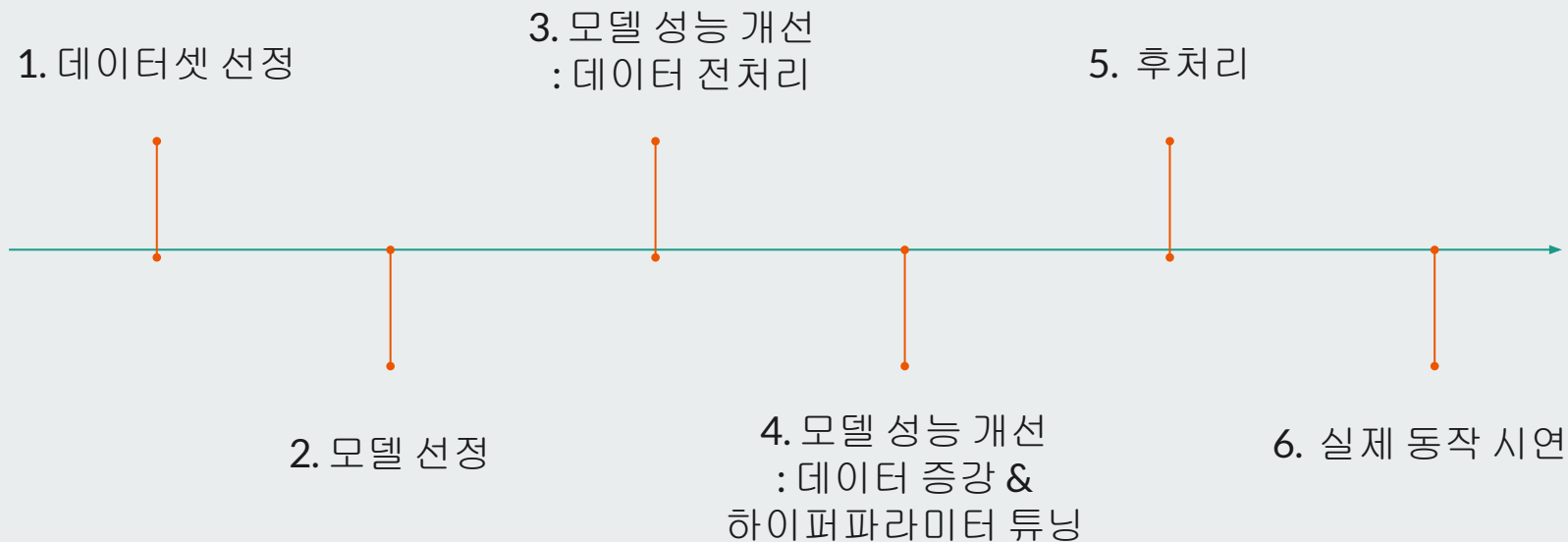
1주차 | 데이터 탐색 & 데이터셋 전처리

2주차 | 데이터 셋을 4개의 장소 카테고리로
나누어

데이터 분석 및 모델 학습 테스트

3주차 | 역할을 분담하여 모델 성능을 개선하고,
감지 및 분류 코드를 작성

Project Overview





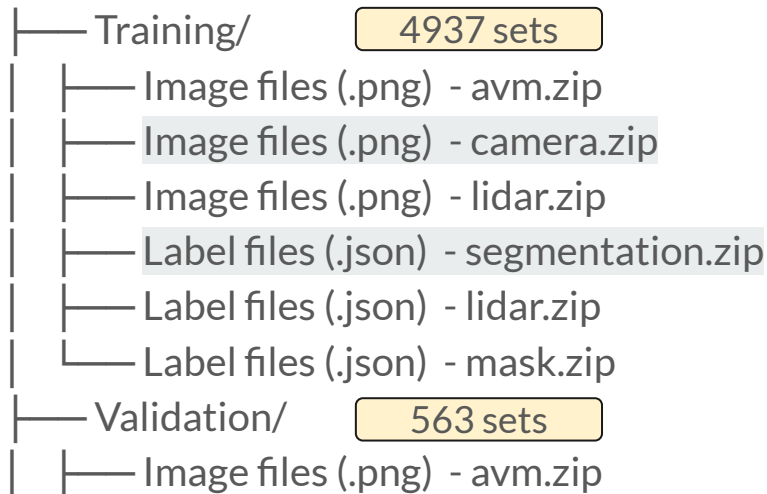
1. 데이터셋 선정

1. 데이터셋 선정



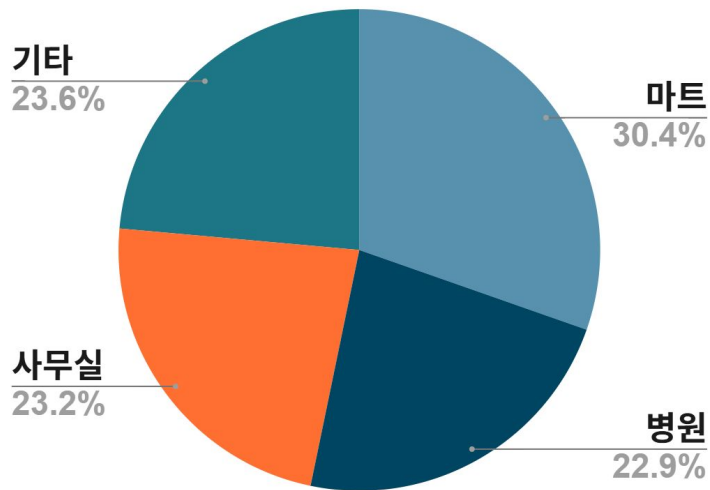
2.67 TB

실내 자율주차용 데이터/

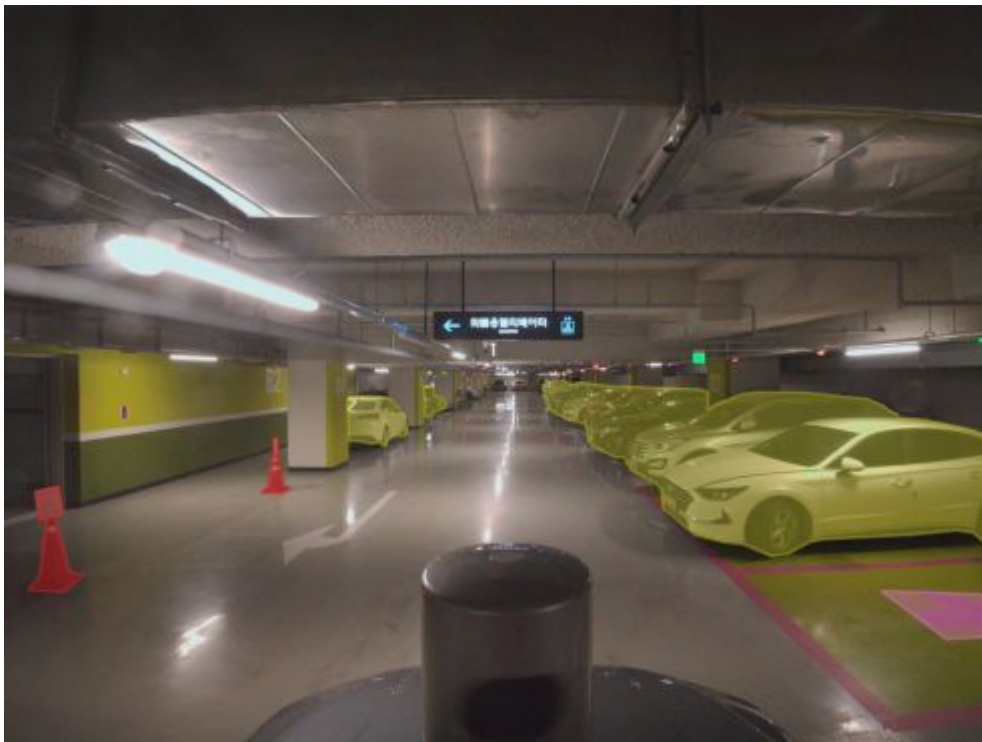


.
. .
.

장소별 데이터 분포 비율



1. 데이터셋 선정



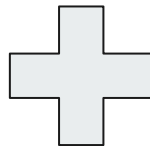
→ Segmentation 라벨이 있어서
객체 분리에 유리할 것으로 판단



2. 모델 선정

2. 모델 선정

모델 탐색



Segformer

- 객체 인식에서 높은 성능을 보인다
- 학습 속도가 빠르다

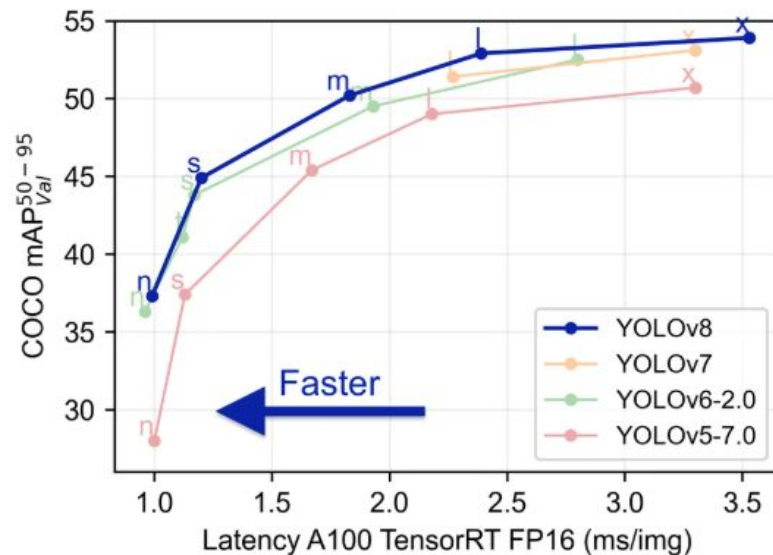
- 영역 인식에 유리하다
- 영역 클래스의 학습을 개선

2. 모델 선정

YOLO



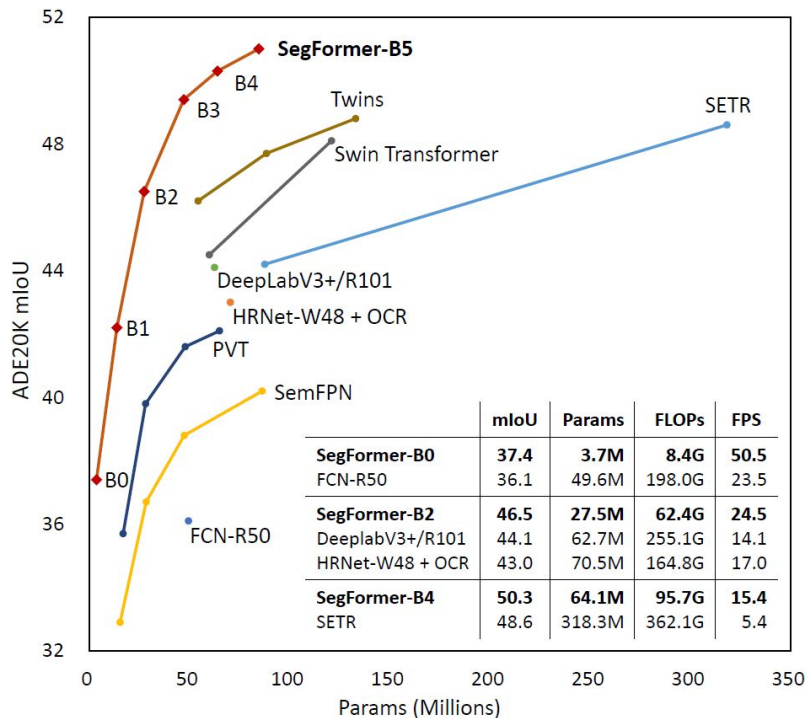
- 객체 인식에서 높은 성능을 보임
- 학습 속도가 비교적 빠름
- 오픈소스를 배포하여 다양한 모델이 존재하며,
학습 효율과 정확도의 절충안을 채택할 수 있음



2. 모델 선정

Segformer

- ❑ Segmentation 모델인 DeepLab V3+, HRNet 등과 비교하여
- ❑ 파라미터 수에 비해 높은 정확도를 가짐
- ❑ 학습 모델의 정확도가 높아질수록, 학습 시간이 증가함



Accurate

2. 모델 선정





3. 모델 성능 개선 : 데이터 전처리

3. 모델 성능 개선 : 데이터 전처리



1. 데이터셋
다운로드

2. 프레임
추출

3. 클래스
선별

4. Label 분류

데이터셋 다운로드

- 코드 모듈화 및 자동화

압축파일 다운로드

임시디렉토리 생성

오류 검증

임시디렉토리 제거

처리 결과 출력 & 로그 기록
→ 데이터 2차 검증
& 코드 실행 시간 분석



실내 자율주차용 데이터/

├ Training/

└ Validation/

├ Image files (.png) - camera.zip

└ Label files (.json) - segmentation.zip



datasets/

├ images (.png)

├ labels (.txt)

├ labels_segformer (.txt)

└ masks (.png)

├ val

└ train

3. 모델 성능 개선 : 데이터 전처리

프레임 추출



❑ 원본 이미지 : 0.5 sec / frame

→ 한정된 용량에서 더 다양한 데이터를 이용하기 위해, 유사도가 높은 프레임을 제거

→ 1 of 4 frames

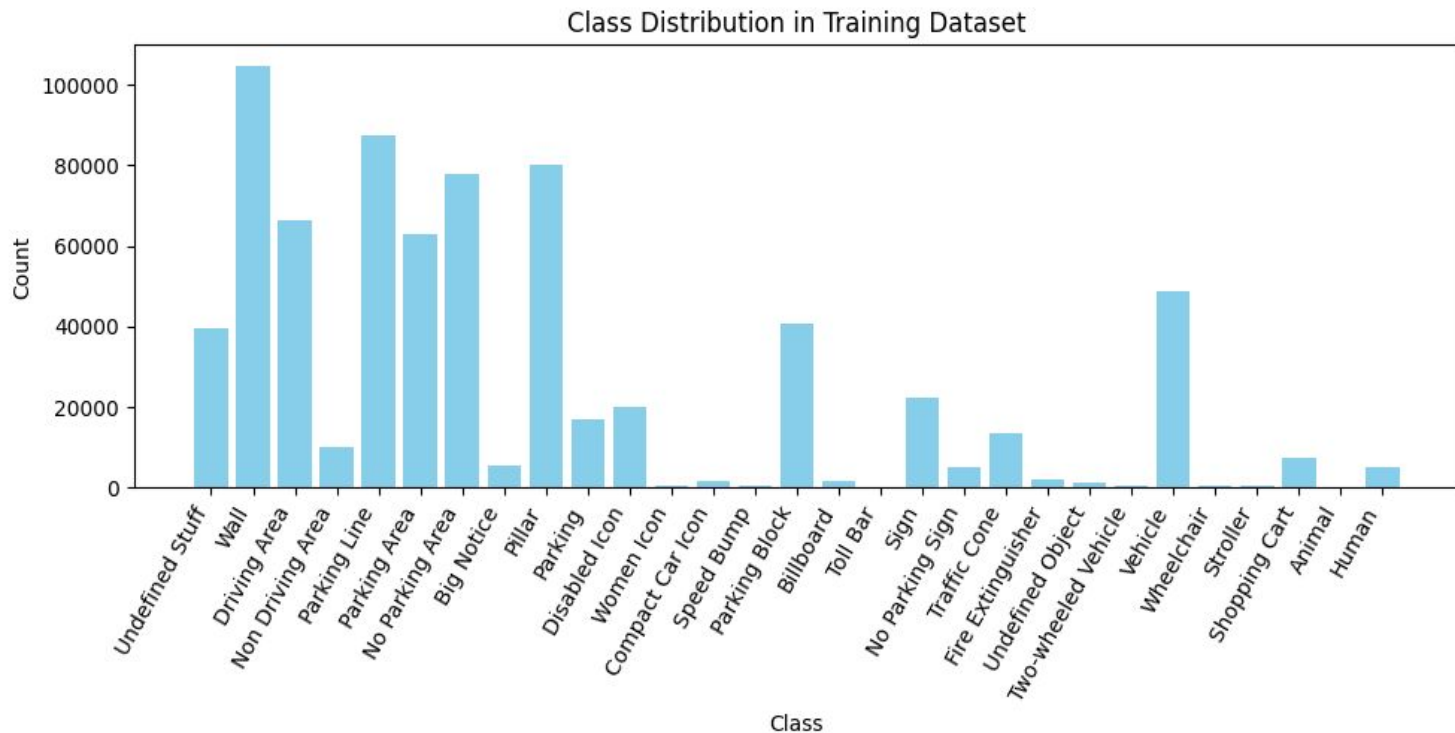


3. 모델 성능 개선 : 데이터 전처리

클래스 분포



- 일부 데이터셋의 클래스 분포 분석
- 클래스가 다양하고 불균형하게 분포하여, 이를 개선하기 위한 시도



YOLO 클래스 병합 시도

- ❑ 데이터가 불균형하게 분포되어 있음 # 유사 아이콘 통합
disabled + women + compact car Icon →
- ❑ 클래스 종류가 많아서 학습 복잡도가 증가함



- ❑ 데이터 수가 적고, 유사한 클래스 병합 # 정보 안내물 통합
Big notice + Billboard + Sign →
 - ❑ 공간 및 객체 인식에 있어 중요도가 낮은 클래스 제거 # 유사 객체 통합
Fire Extinguisher + Animal →
two-wheel + Vehicle →
Stroller + Shopping Cart + Wheelchair →
- 데이터 불균형은 완화되었으나, 인식을 저하 발생

```
nc: 16
names:
  0: 'Undefined Stuff'
  1: 'Wall'
  2: 'Driving Area'
  3: 'Non Driving Area'
  4: 'Parking Line'
  5: 'Parking Area'
  6: 'No Parking Area'
  7: 'Signage'
  8: 'Pillar'
  9: 'No Parking Icon'
  10: 'No Parking Sign'
  11: 'Traffic Cone'
  12: 'Undefined Object'
  13: 'Vehicle'
  14: 'Cart-like Object'
  15: 'Human'
```

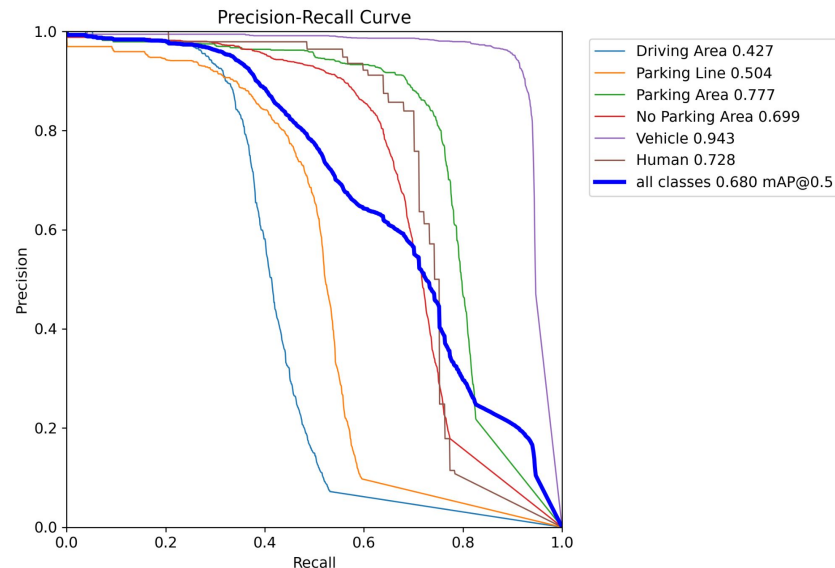
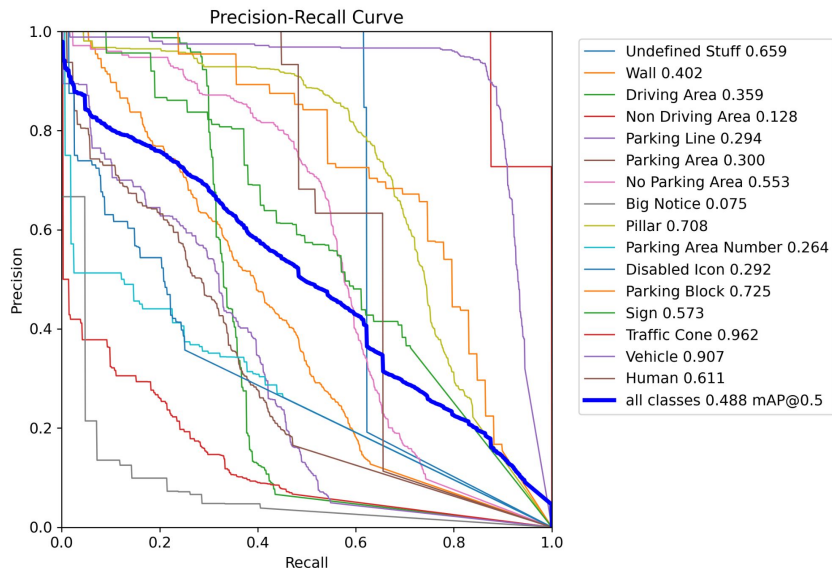
YOLO 성능 개선 시도

항목	1차 실험	2차 실험	3차 실험	4차 실험
 데이터 수	약 300장	약 300장	약 1800장	약 1800장
 클래스 수	29	29	6	6
 모델	YOLOv8n-seg	YOLOv8m-seg	YOLOv8n-seg	YOLOv8n-seg
 해상도	640	1024	640	1024
 mAP@0.5	0.467	0.559	0.606	0.680

- ❑ 모델 성능 개선 시도 : 클래스 감축, 모델 버전 변경, 해상도 up
- ❑ 클래스를 줄이거나, YOLO 의 더 높은 모델을 사용하여도 정확도의 한계가 존재
- ➔ 정확도가 낮은 **영역 감지**에 다른 모델을 채택

3. 모델 성능 개선 : 데이터 전처리

[Trouble shooting - 진나영]



- ❑ 모델 성능 개선 시도 : 클래스 재선별, 해상도/모델 버전 변경
- ❑ 클래스를 줄이거나, YOLO 의 더 높은 모델을 사용하여도 정밀도의 한계가 존재
- ➔ 정확도가 낮은 영역 감지에 다른 모델을 채택

3. 모델 성능 개선 : 데이터 전처리

최종 클래스 선별

- 클래스의 특성과 인식 목적에 따라 클래스를 분리하여 학습

YOLO - 객체 인식 클래스 (13)

특별주차공간

0: Disabled Icon
1: Women Icon
2: Compact Car Icon

주차 불가

3: No Parking Sign
4: Traffic Cone

장애물

5: Fire Extinguisher
6: Undefined Object

이동물체

7: Two-wheeled Vehicle
8: Vehicle
9: Wheelchair
10: Stroller
11: Shopping Cart
12: Human

Segformer - 영역 인식 클래스 (2)

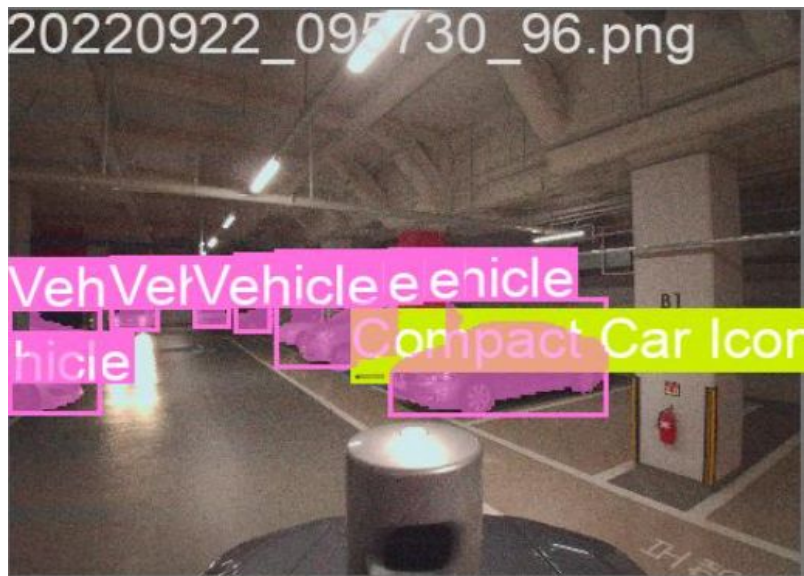
0: Driving Area
1: Parking Area

3. 모델 성능 개선 : 데이터 전처리

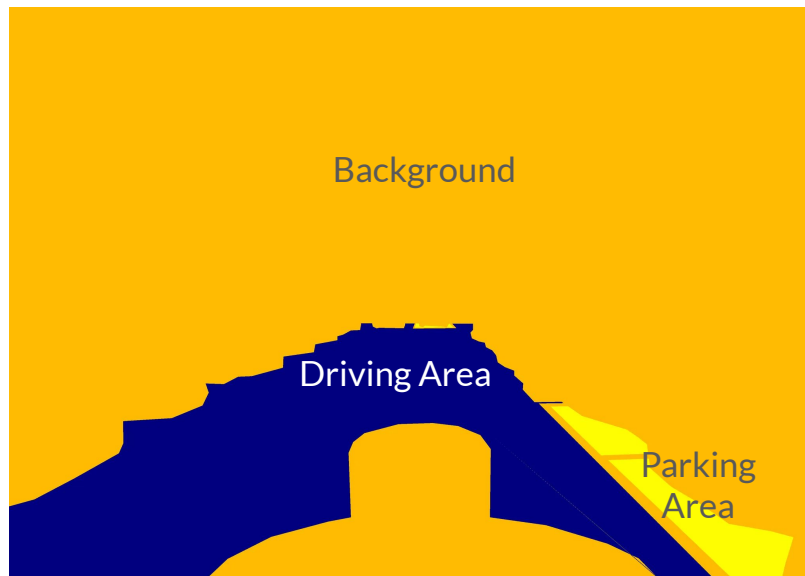
라벨 분류



YOLO label - 객체 인식

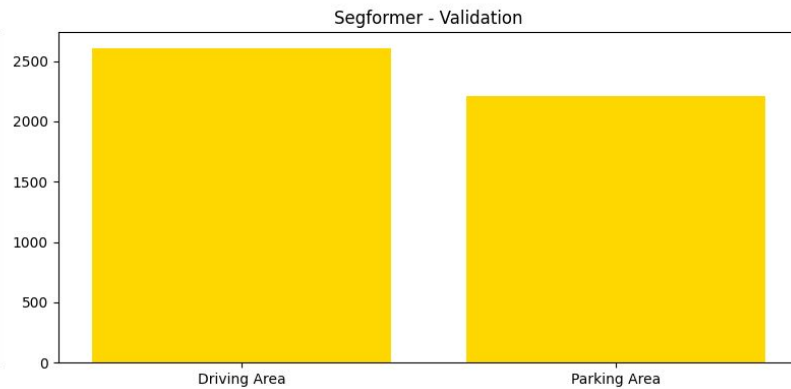
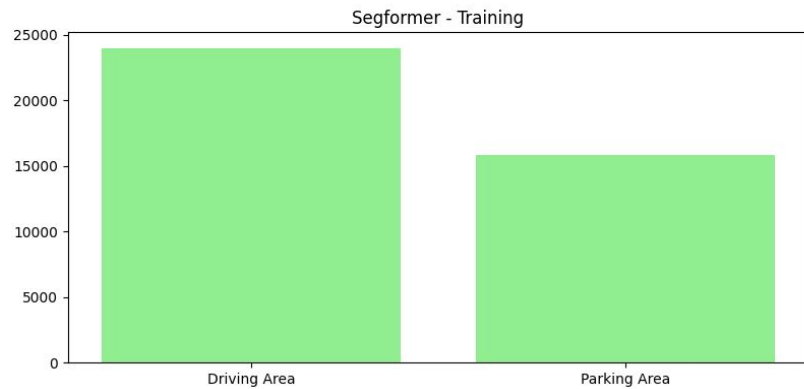
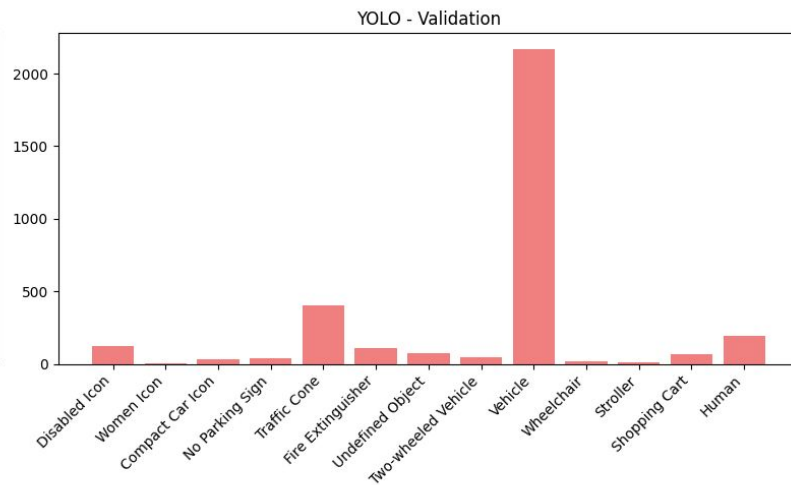
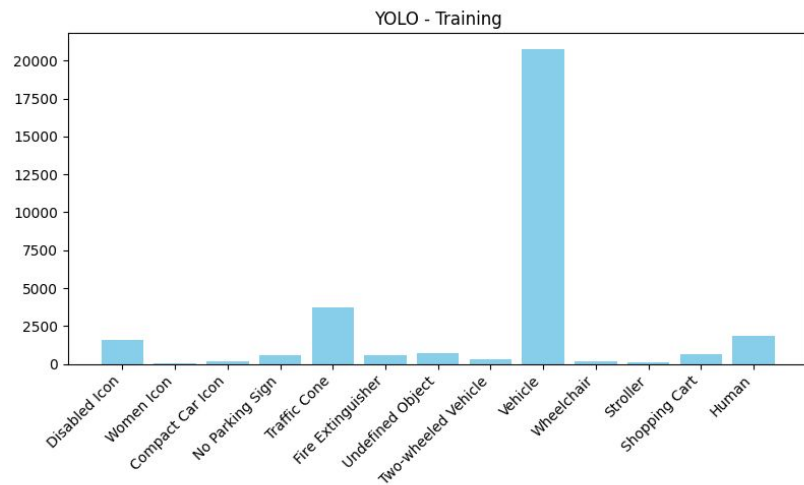


Segformer label - 영역 인식



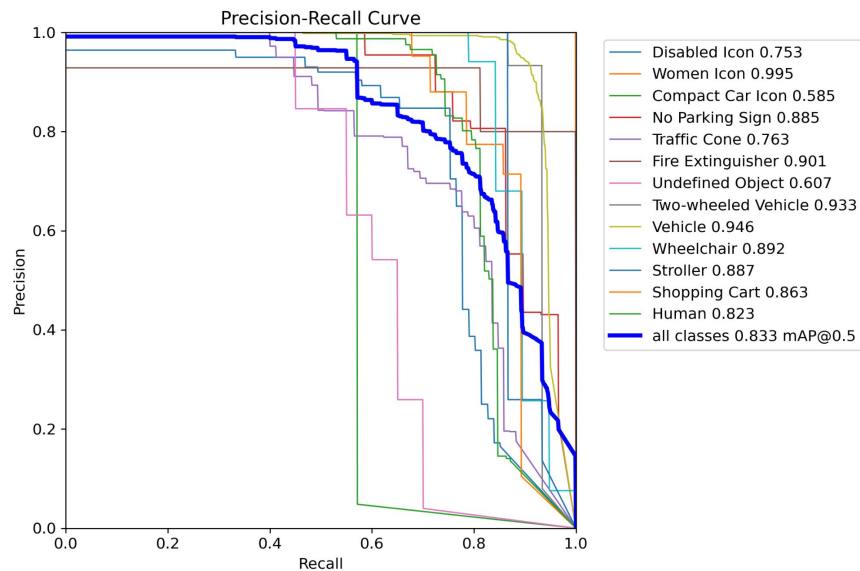
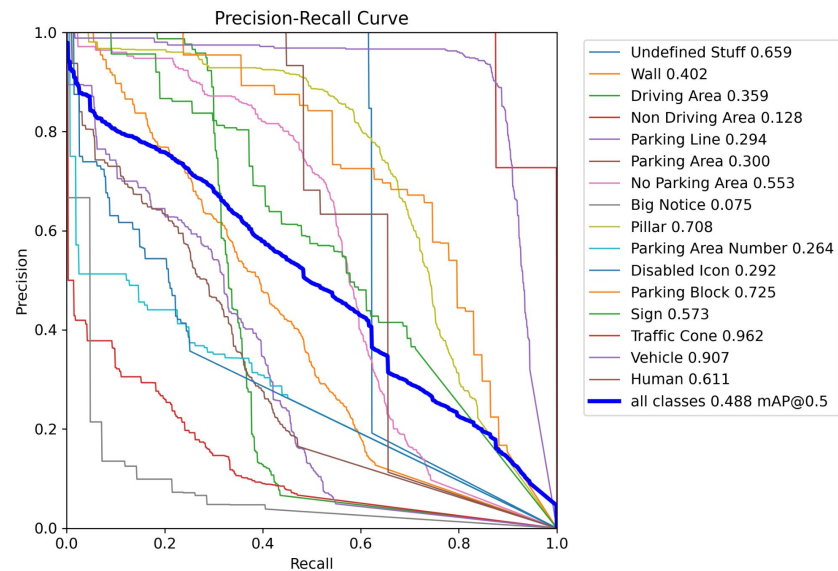
3. 모델 성능 개선 : 데이터 전처리

Class Distribution



3. 모델 성능 개선 : 데이터 전처리

YOLO - class 조절

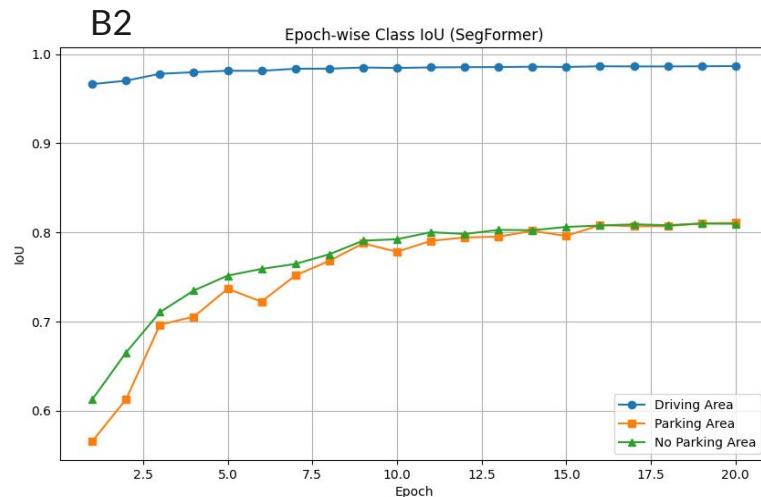
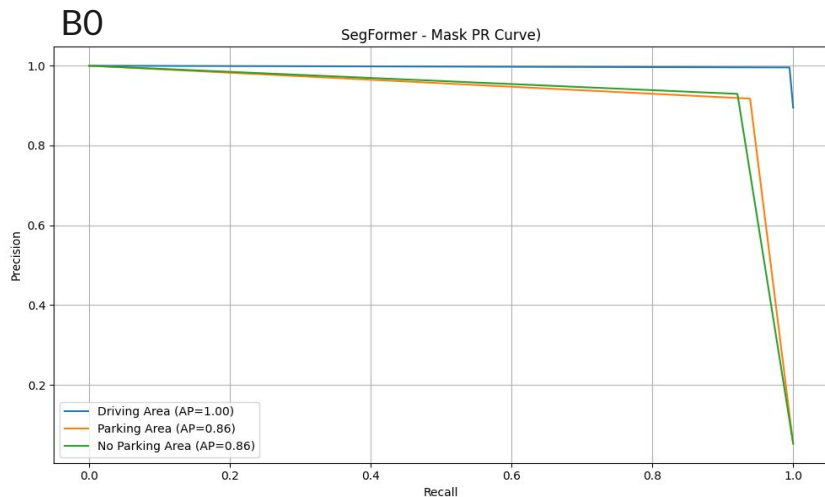


□ 영역 클래스 분리후, 단순히 클래스를 제거하였을 때보다 높은 성능을 보임



4. 모델 성능 개선 : 데이터 증강 & 하이퍼파라미터 튜닝

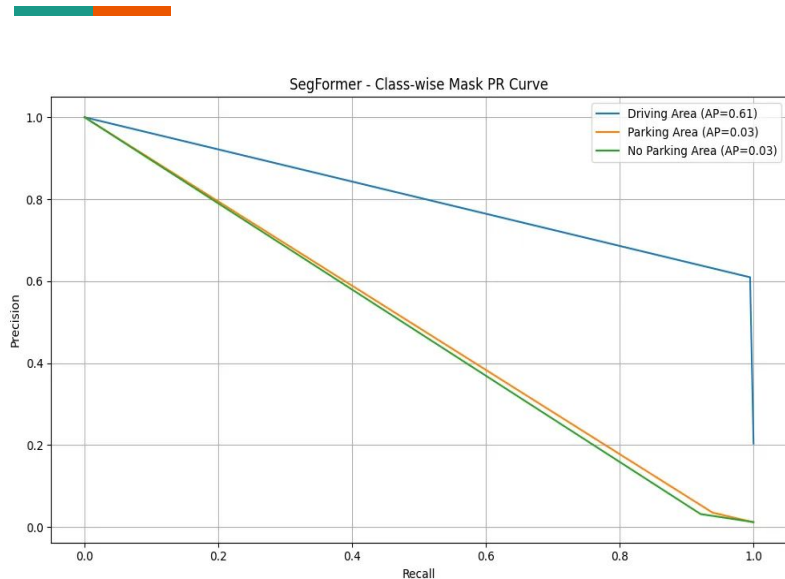
Segformer



❑ Driving / Parking / No Parking Area 감지시, Segformer B0 / B2 둘 다 괜찮은 성능을 보임

➔ 시간 효율적인 B0 모델을 채택

Segformer - Background 무시



```
id2label = {0: "Driving Area", 1: "  
num_classes = len(id2label)  
ignore_index = 255
```

```
valid_mask = mask != ignore_index  
mask = mask[valid_mask]  
pred = pred[valid_mask]
```

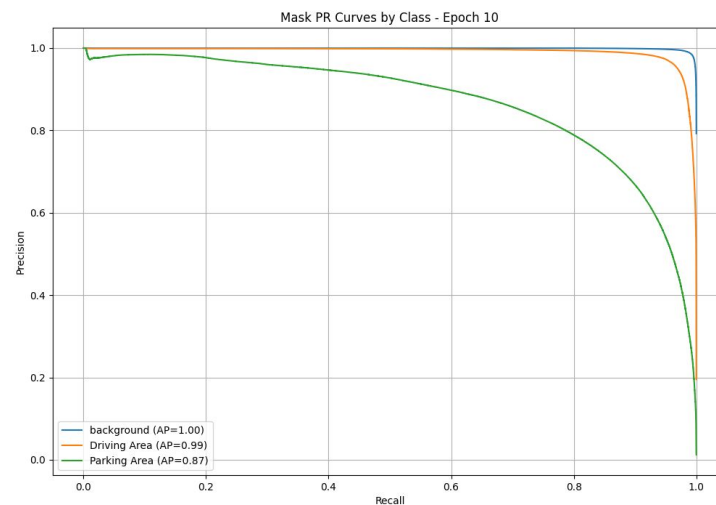
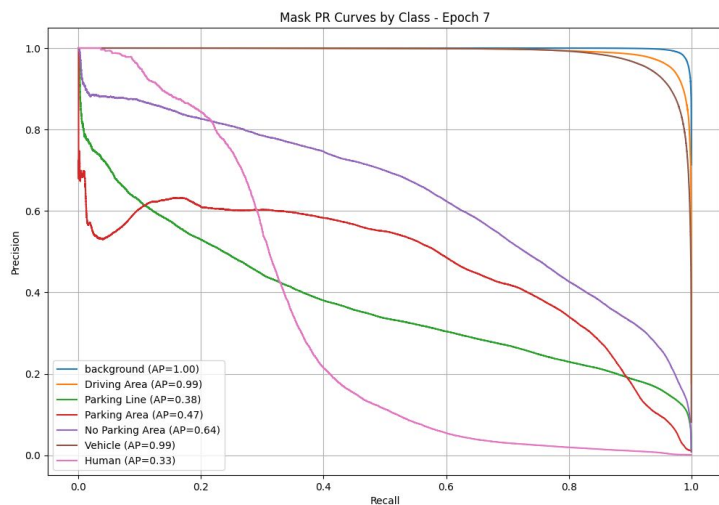
```
all_labels.append(mask.flatten())  
all_preds.append(pred.flatten())
```

- ❑ 성능 평가 과정에서 문제가 발생
- ➔ 평가 지표로 계산하는 과정에서 **Background(=255)**를 같이 **flatten** 하여 오류 발생

➔ 255 무시 처리하여 해결

Segformer - batch, class 조절

❑ Object 클래스 제거 후 배치 사이즈 조절



- train batch size = 4
- number of class = 6

- train batch size = 16
- number of class = 2

YOLO 성능 저하를 해소하기 위한 시도

❑ Over / under sampling

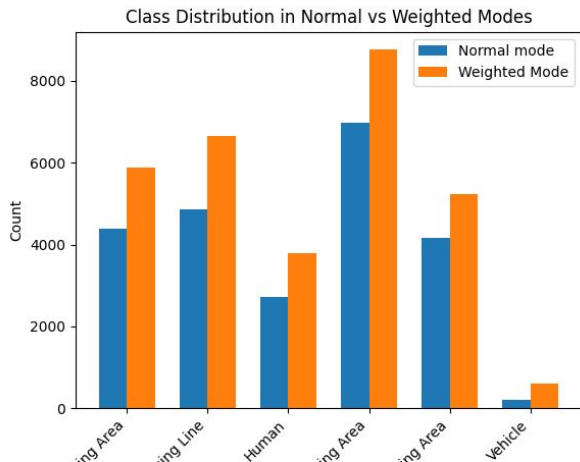


- ❑ 하나의 이미지에 다수,소수 클래스가 동시에 라벨링되어 샘플링의 효과가 미미함
- 소수 클래스만 **crop** 하여 데이터 증강 후 랜덤 배경에 합성하여 학습
- 과정이 번거롭고 랜덤 배경이라는 요소가 학습성능에 악영향을 끼칠 가능성

YOLO 성능 저하를 해소하기 위한 시도



❑ 클래스 가중치 조절



❑ YOLO - [loss.py](#) 수정하여 데이터 양이 적은 클래스의 손실함수에 가중치를 둠

➔ 기존의 소스파일 구조가 복잡하여 수정 시 리스크 발생

❑ Monkey Patch - class weight

➔ 데이터셋이 학습 세션으로 진입하는 과정에서 소수 클래스의 가중치를 높이는 중간단계 추가

➔ YOLO 내부의 가중치 알고리즘과 충돌하여 유의미한 가중치를 적용하지 못함

4. 모델 성능 개선

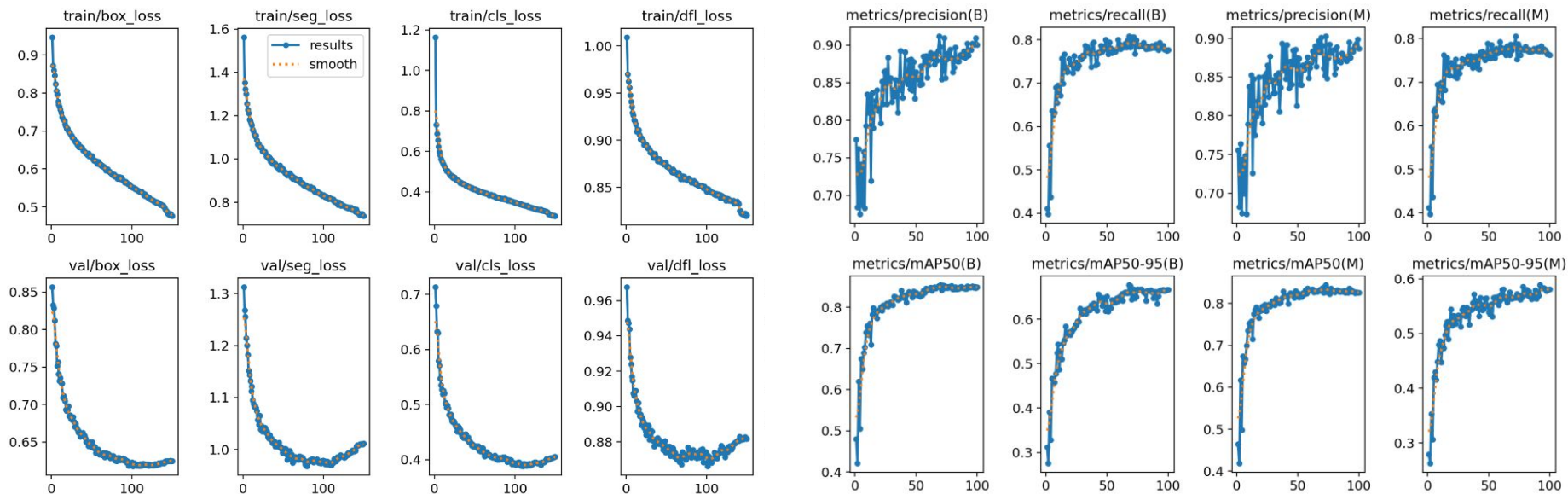
YOLO 성능 저하를 해소하기 위한 시도



❑ 프레임워크 변경 및 클래스 감축

- YOLO 는 Object 검출 기반 프레임워크,
Segformer 를 이용하여 Area 검출
- 학습된 두 프레임워크를 병합하여 영상을
인식하는 프로그램 작성

YOLO - Epoch 조절



□ Epoch = 100 이후에 과적합되는 추세

Hyperparameter 튜닝



Model train 방식

- 학습시 model, epoch, imgsz, batch 를 조절하는 방법
- img size 1088, batch size 16, epochs 수는 처음 학습 시 50~70, pre-trained 가중치 사용 시 30~50 에서 가장 효율적인 학습 성능을 보임

Ray tune 방식

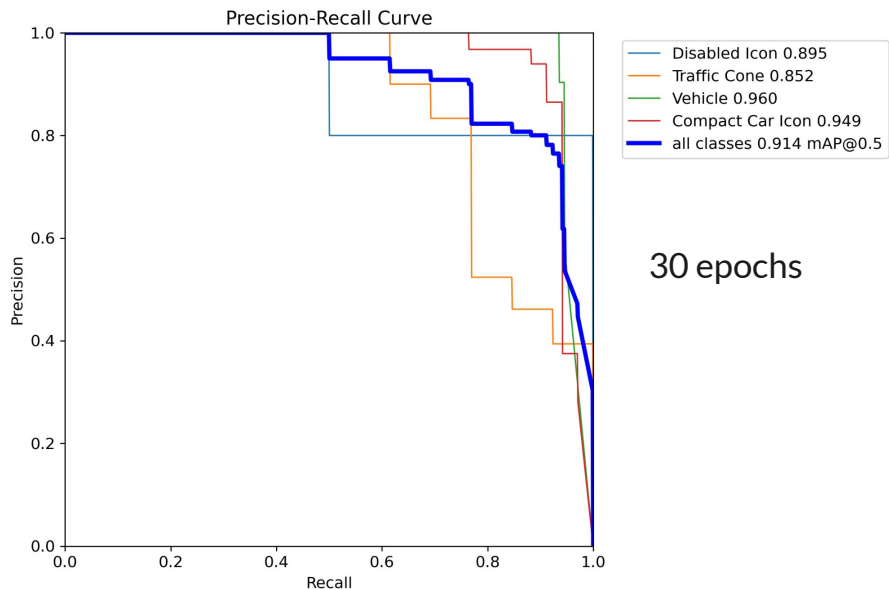
- 하이퍼파라미터를 자동 탐색하여 미세 튜닝
- 데이터가 충분하여 인식률이 극적으로 향상되지 않음

Image datasets 추가 학습

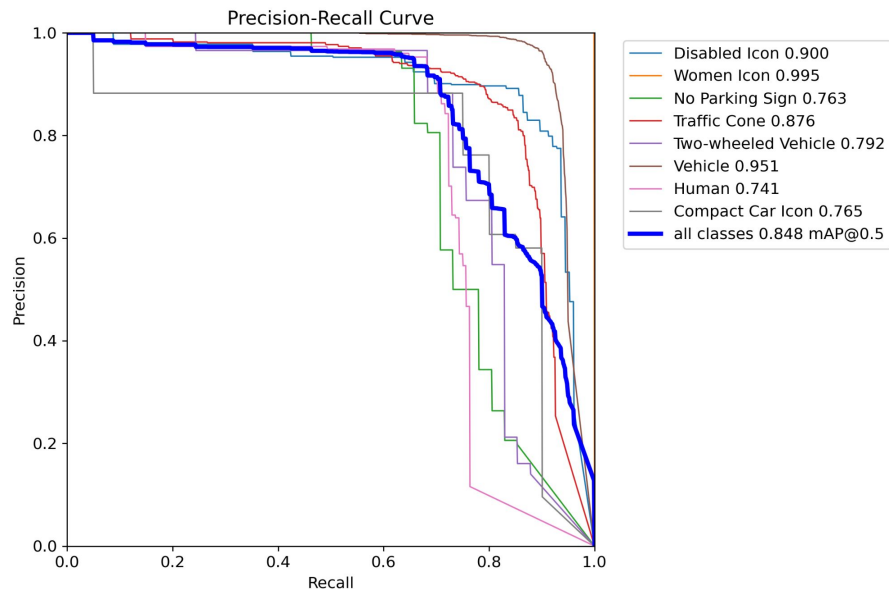
- mAP 값이 낮은 라벨의 데이터를 보강하기 위해 기존 데이터 (약 4000장)에 COCO 데이터(약 1000장) 을 추가로 학습시킴
- 1차 시도 : COCO datasets 의 ‘오토바이’, ‘사람’ 이미지 추가학습
- 2차 시도 : ‘Compact car Icon’ 라벨 포함된 이미지 증강하여 추가학습

Compact Car Icon 추가학습

□ 추가 학습 모델



□ 추가 학습 결과



→ 학습에서 경차 아이콘의 정밀도가 낮게 나와서 해당 라벨을 포함한 이미지를 증강하여 추가 학습



5. 후처리

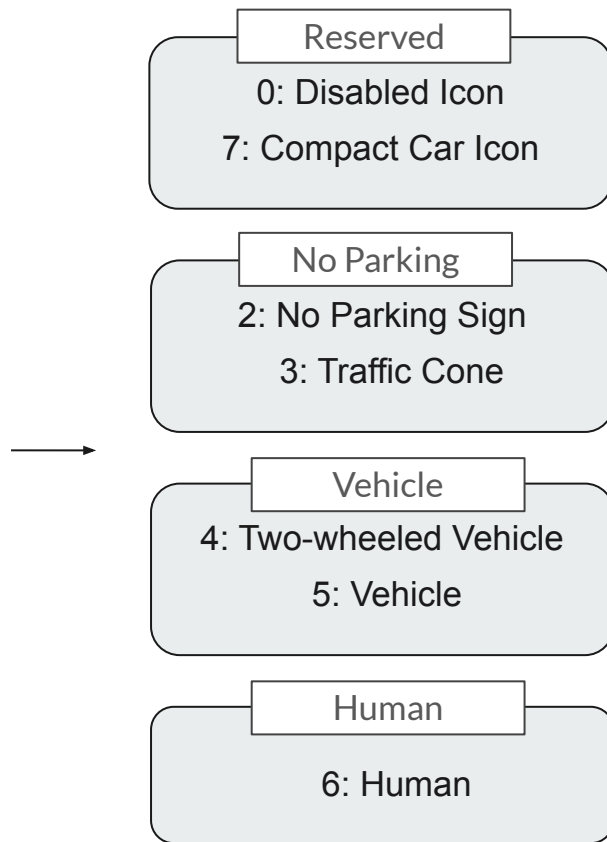
5. 후처리

후처리 - Yolo

```
# 클래스 이름 및 범주 설정
class_names = [
    "Disabled Icon", "Women Icon", "No Parking Sign", "Traffic Cone",
    "Two-wheeled Vehicle", "Vehicle", "Human", 'Compact Car Icon'
]

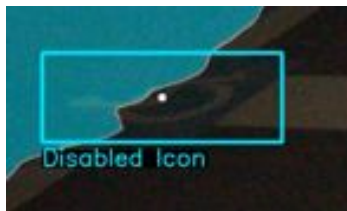
object_categories = {
    "Reserved": [0, 7],
    "NoParking": [2, 3],
    "Vehicle": [4, 5],
    "Human": [6]
}
```

❑ 클래스별 카테고리 분류



5. 후처리

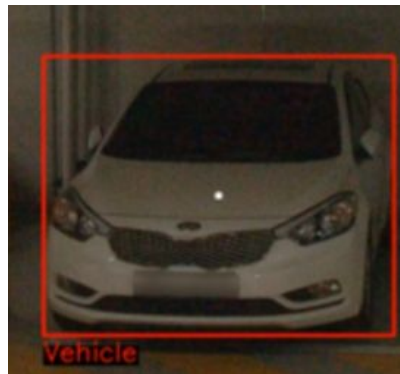
후처리 - Yolo



Reserved



NoParking



Vehicle



Human

→ 객체 바운딩 박스 ROI, 하단에 라벨 표시, 중심점(하얀색)

5. 후처리

후처리 - Segformer

학습 Class ID

```
BACKGROUND_ID = 0
DRIVING_AREA_ID = 1
PARKING_AREA_ID = 2
```

주차공간 Color

```
parking_status_colors = {
    "Empty": (100, 255, 100),
    "Caution": (0, 140, 255),
    "Reserved": (255, 255, 0),
    "Blocked": (0, 0, 200)
}

segformer_colors = {
    0: (0, 0, 0),
    1: (200, 150, 255),
}
```

카테고리 재분류

```
# 일반 카테고리 분류 기준
if category == "Reserved":
    region_status = "Reserved"
elif category == "Human":
    region_status = "Caution"
elif category in ["NoParking", "Vehicle"]:
    region_status = "Blocked"
```

* mask 가 반투명하여, 원본과 mask 색이
3:7 로 혼합되어 다른 색으로 보일 수 있음

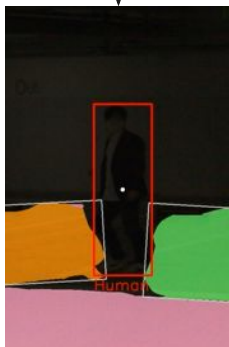
주차상태 시각화

```
for c in range(3):
    color_mask[:, :, c][region_mask == 1] = parking_status_colors[region_status][c]

contours, _ = cv2.findContours(region_mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
cv2.drawContours(color_mask, contours, -1, (255, 255, 255), 2)

overlay = cv2.addWeighted(frame, 0.3, color_mask, 0.7, 0)
edge_map = cv2.Canny(preds.astype(np.uint8), 50, 150)
edges_rgb = cv2.cvtColor(edge_map, cv2.COLOR_GRAY2BGR)
edges_rgb[np.where((edges_rgb != [0, 0, 0]).all(axis=2))] = [255, 255, 255]
final_overlay = cv2.addWeighted(overlay, 1.0, edges_rgb, 0.7, 0)
```

후처리 - 주차 영역 분류



[Trouble Shooting]

- ❑ NoParking 객체는 주차 공간 안보다 앞에 위치한 경우가 많음
- ❑ Human은 주차 공간과 조금만 겹쳐도 주의해야 함



- IOU(box/region) 허용 조건을 추가
- IOU조건(>0.01) 조절

후처리 - 주차 영역 분류

```
x1, y1, x2, y2 = map(int, box_data[:4])
cx, cy = int((x1 + x2) / 2), int((y1 + y2) / 2)

in_region_by_center = (cy < h and cx < w and clipped_region[cy, cx] == 1)

iou_condition = False
if category in ["NoParking", "Human"]:
    box_mask = np.zeros((h, w), dtype=np.uint8)
    box_mask[y1:y2, x1:x2] = 1
    intersection = np.logical_and(clipped_region, box_mask).sum()
    region_area = clipped_region.sum()
    box_area = (x2 - x1) * (y2 - y1)
    if box_area > 0 and (intersection / box_area > 0.01):
        iou_condition = True
    elif region_area > 0 and (intersection / region_area > 0.01):
        iou_condition = True

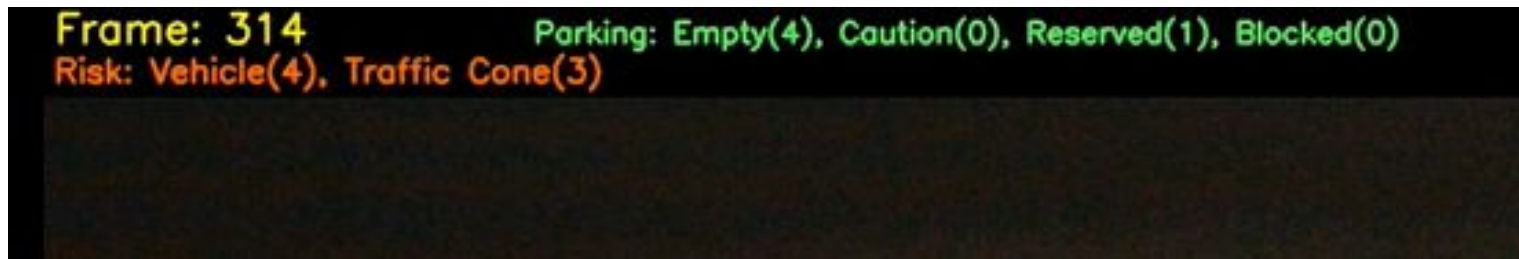
if not (in_region_by_center or iou_condition):
    continue
```

□ 영역 내 객체 판단 기준

- 일반 객체: 중심점이 주차 공간 내부에 위치
- **NoPaking, Human 객체:** 중심점 조건 or IOU 조건 만족

5. 후처리

HUD 상태창

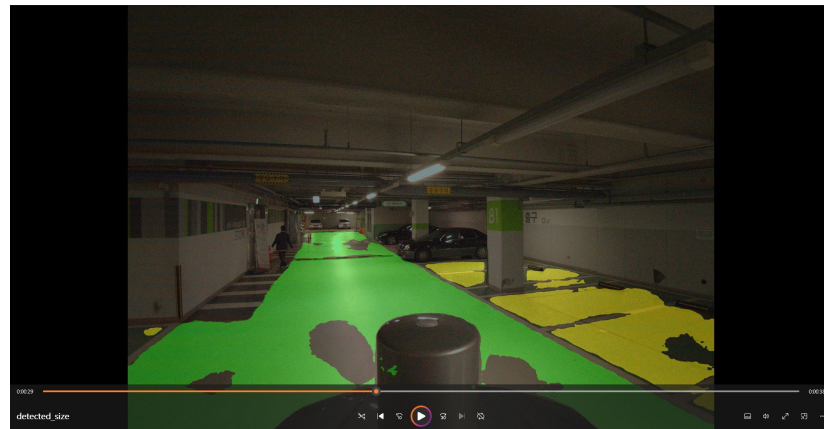


- 영상 상단에 객체&영역 탐지 상태창 표시
 - Frame 번호
 - 주차 공간 분류 요약
 - 객체 탐지 요약



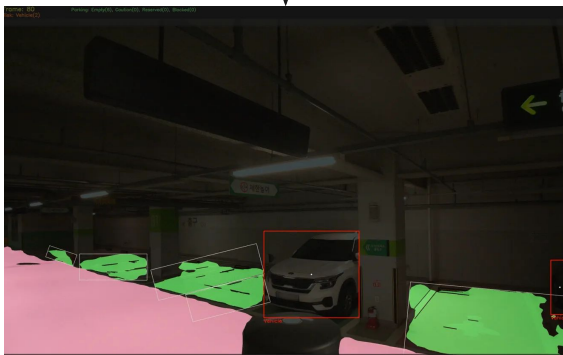
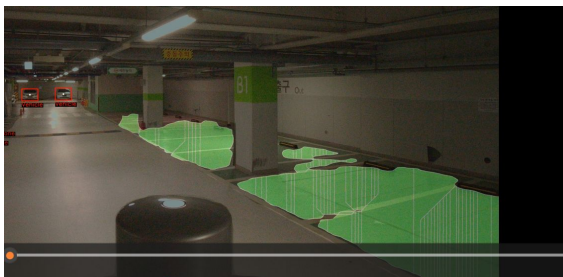
6. 실제 동작 시연

인식률 향상



- 해상도 조절 512x512 이상에서는 인식률이 감소하는 경향
- 시각화 처리 (경계 노이즈 제거, 경계 또렷하게)

인식을 향상



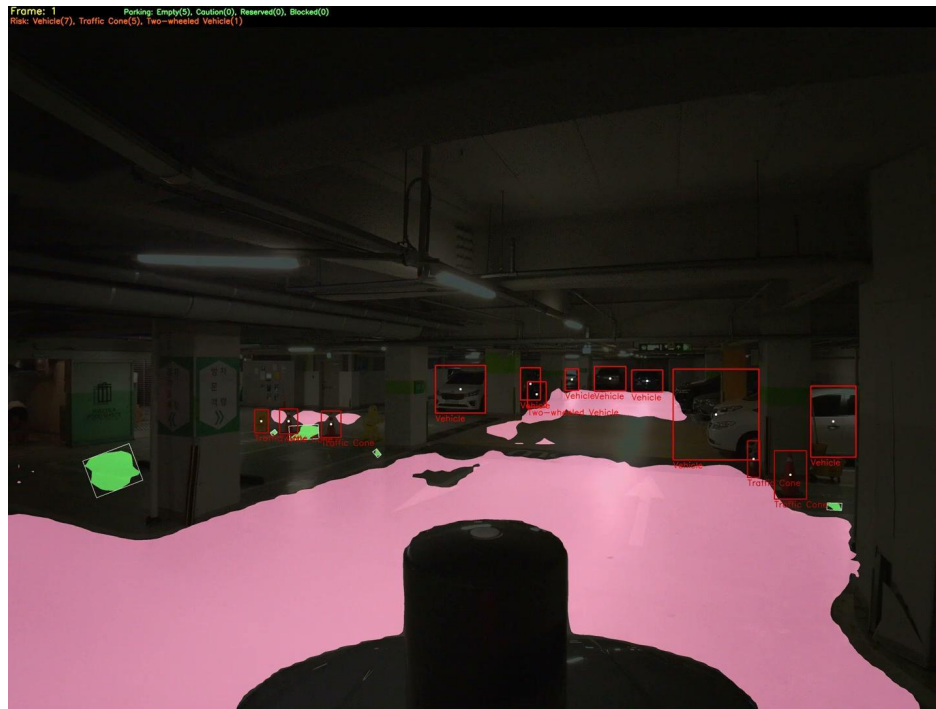
❑ 붙어있는 주차공간 분할이 안되는 문제

→ 단순 `scipy.ndimage.label()` 사용

- ❑ Canny + HoughLinesP → parking_area 내 주차선(검정) 검출
- ❑ Connected Components → 주차 공간 후보 영역 분리
- ❑ minAreaRect → 각 영역을 회전 사각형으로 감싸고,
Polygon 생성
- ❑ 면적 & 종횡비 필터링 → 잡음 제거
- ❑ 좌표 클리핑 → 화면 넘침 방지

6. 실제 동작 시연

Train1



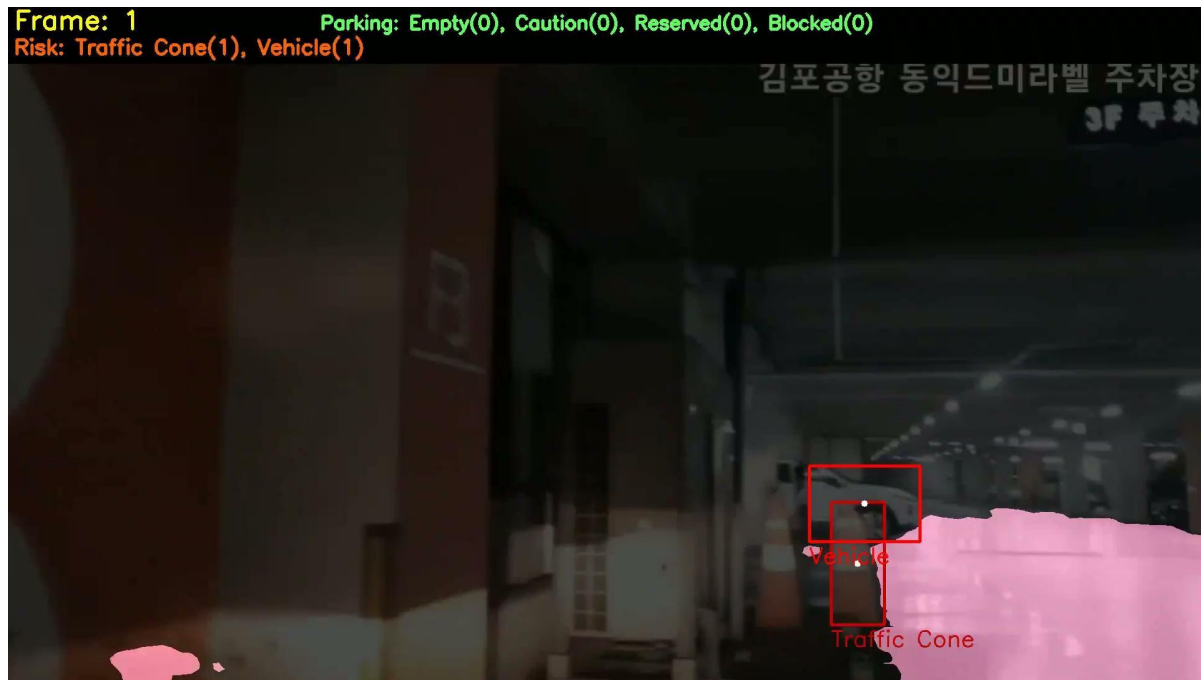
6. 실제 동작 시연

Train2



6. 실제 동작 시연

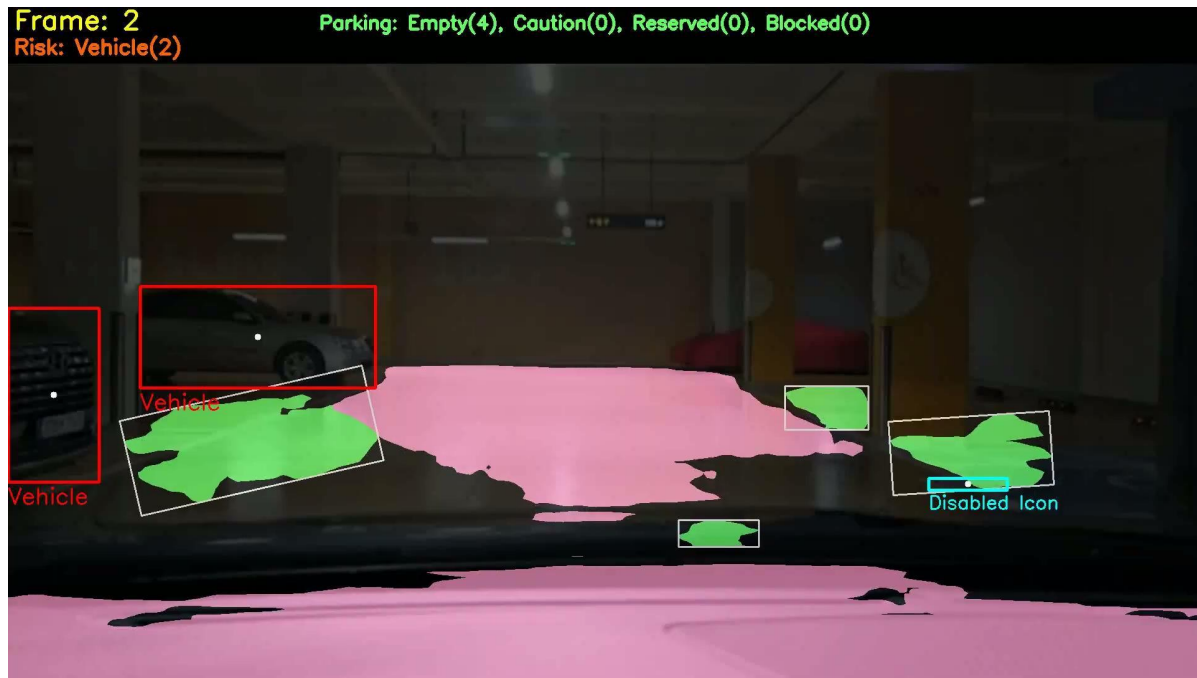
Sample1



<https://youtu.be/CQNHITP3yR0?si=x6WC037KrCA-GLC->

6. 실제 동작 시연

Sample2



<https://youtu.be/m970AV1Co-o?si=iSmUz9y8BZimvISm>

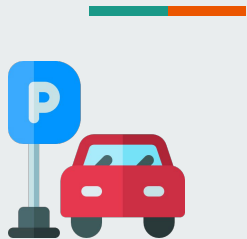
김소원 | 일방향 이론 수업을 듣다가 프로젝트를 통해 팀원들과 코치님과 소통하며 모델을 만드는 과정을 경험하면서 실무적으로 많이 배울 수 있었다. 중간 프로젝트를 마치며 다시 시작되는 이론 수업에서 더 성실히 학습해야겠다고 다짐하였다.

서영훈 | YOLO 모델을 직접 구축하고 학습시키는 과정은 쉽지 않았지만, 그만큼 보람 있는 경험이었습니다. 처음에는 데이터셋을 선정하고 데이터 분석하는 것부터 쉽지 않았고, 학습 결과를 보며 왜 결괏값이 낮은지 고민하면서 하이퍼파라미터 조정, 데이터 증강, COCO 데이터셋을 추가로 학습시키는 등 많은 시행착오를 겪었습니다. 이 과정에서 데이터의 양보다 질이 더 중요하다는 것을 깨달았으며, 이 경험을 토대로 앞으로는 데이터 전처리와 품질 관리에 더욱 신경 쓰고, 다음에는 상황에 맞는 최적의 모델을 설계하여 대회에서 좋은 결과를 얻고 싶습니다.

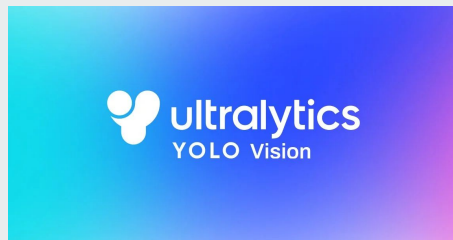
전호준 | SegFormer와 YOLO를 결합한 이번 프로젝트는, 데이터 부족과 클래스 불균형, 환경 차이에 따른 성능 저하 등 다양한 문제를 해결해가며 점진적으로 모델 성능을 개선해나간 과정이었다. 백본 freeze, 샘플링 기법, 클래스 가중치 적용, 모델 분리 적용 등 실험을 반복하며 최적의 방식에 다가가는 과정을 통해, 단순한 학습을 넘어 실제 데이터 상황에 유연하게 대응하는 경험을 할 수 있었고, 이로부터 모델 학습의 깊이와 데이터 이해력을 함께 키울 수 있었다.

진나영 | 반나절씩 비대면으로 진행된 프로젝트라 소통과 집중 면에서 어려움이 있었지만, 자율주행 AI 개발 전 과정을 직접 경험하며 실무 감각을 키울 수 있었다. 데이터 선정부터 EDA, 모델 학습, 후처리, 시연 영상, 협업 툴 사용까지 차근차근 쌓아가며 문제를 해결했고, 다음에는 더 명확한 목표 아래 완성도 높은 결과를 만들고 싶다.





YOLO



Project Overview

1. 데이터셋 선정

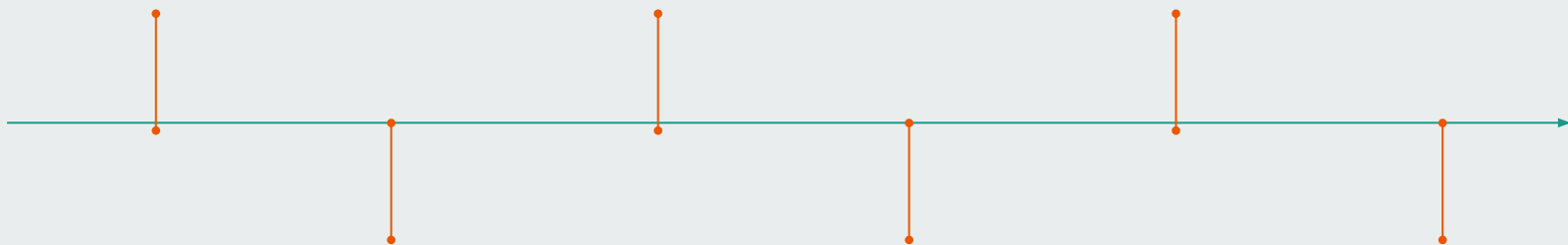
3. 모델 선정

5. 후처리

2. 데이터 전처리

4. 모델 성능 테스트 &
파인튜닝

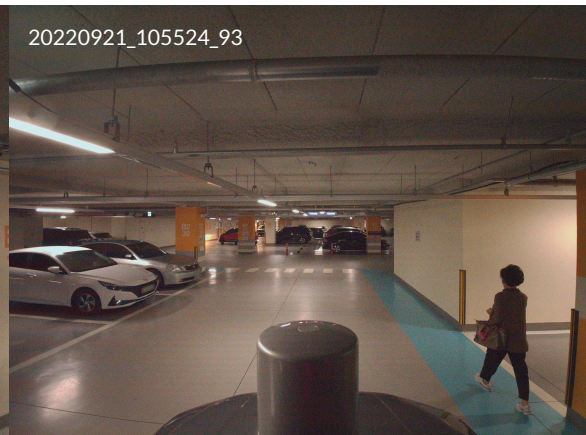
6. 실제 동작 시연



2. 데이터 전처리

프레임 추출

- 1 per 4 frames



5. 후처리

후처리 - Segformer

```
BACKGROUND_ID = 0
DRIVING_AREA_ID = 1
PARKING_AREA_ID = 2

parking_status_colors = {
    "Empty": (100, 255, 100),
    "Caution": (0, 140, 255),
    "Reserved": (255, 255, 0),
    "Blocked": (0, 0, 200)
}

segformer_colors = {
    0: (0, 0, 0),
    1: (200, 150, 255),
}
```

```
# 일반 카테고리 분류 기준
if category == "Reserved":
    region_status = "Reserved"
elif category == "Human":
    region_status = "Caution"
elif category in ["NoParking", "Vehicle"]:
    region_status = "Blocked"
```

```
for c in range(3):
    color_mask[:, :, c][region_mask == 1] = parking_status_colors[region_status][c]

contours, _ = cv2.findContours(region_mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
cv2.drawContours(color_mask, contours, -1, (255, 255, 255), 2)

overlay = cv2.addWeighted(frame, 0.3, color_mask, 0.7, 0)
edge_map = cv2.Canny(preds.astype(np.uint8), 50, 150)
edges_rgb = cv2.cvtColor(edge_map, cv2.COLOR_GRAY2BGR)
edges_rgb[np.where((edges_rgb != [0, 0, 0]).all(axis=2))] = [255, 255, 255]
final_overlay = cv2.addWeighted(overlay, 1.0, edges_rgb, 0.7, 0)
```

❑ 학습 class_id:

배경(0), 주행 공간(1), 주차 공간(2)

❑ 주차 공간:

빈 주차 공간: 연두색

사람이 있는 주차 공간: 하늘색

특별 주차 공간: 노란색

주차 불가능: 빨간색

윤곽선: 흰색

❑ 배경: 검정(무시 처리)

❑ 주행 공간: 연보라

+) 반투명 mask 이기 때문에, 원본 색과 mask 색이 섞여(3:7) 다른 색으로 보일 수도 있음